Parking Lot System - Complete LLD Interview Guide

Interview Duration: 45 minutes | Difficulty: Medium-High | Must-Know: ⭐⭐⭐

🎯 WHAT TO ACTUALLY WRITE IN INTERVIEW (20 mins coding)

✅ MUST WRITE ON WHITEBOARD/SCREEN:

1. Abstract Base Class - ParkingSpot (~5 mins)

```
public abstract class ParkingSpot {
    private String spotId;
    private Vehicle vehicle;
    private SpotType spotType;

    public abstract boolean canFitVehicle(Vehicle vehicle);

    public synchronized boolean assignVehicle(Vehicle vehicle) {
        if (!canFitVehicle(vehicle)) return false;
        this.vehicle = vehicle;
        return true;
    }
}
```

2. ONE Concrete Example - CompactSpot (~3 mins)

```
public class CompactSpot extends ParkingSpot {
    @Override
    public boolean canFitVehicle(Vehicle vehicle) {
        return vehicle.getType() == VehicleType.CAR ||
               vehicle.getType() == VehicleType.BIKE;
    }
}
```

3. Strategy Interface - ParkingStrategy (~2 mins)

```
public interface ParkingStrategy {
    ParkingSpot findSpot(Vehicle vehicle, List<Level> levels);
}
```

4. Main Facade - ParkingLot (~10 mins)

```
public class ParkingLot {
    private static ParkingLot instance;
    private List<Level> levels;
    private Map<String, Ticket> activeTickets;
    private ParkingStrategy parkingStrategy;

    public static synchronized ParkingLot getInstance() {
        if (instance == null) {
            instance = new ParkingLot();
        }
        return instance;
    }

    public synchronized Ticket parkVehicle(Vehicle vehicle) {
        ParkingSpot spot = parkingStrategy.findSpot(vehicle, levels);
        if (spot == null) return null;

        spot.assignVehicle(vehicle);
        String ticketId = UUID.randomUUID().toString();
        Ticket ticket = new Ticket(ticketId, vehicle, spot);
        activeTickets.put(ticketId, ticket);
        return ticket;
    }

    public synchronized double unparkVehicle(String ticketId) {
        // Write signature, explain: "Calculate fee using PricingStrategy"
        // "Release spot by calling spot.removeVehicle()"
    }
}
```

🗣️ **EXPLAIN VERBALLY (Don't write full code):**

- "LargeSpot, BikeSpot, HandicappedSpot follow same pattern as CompactSpot"
- "Vehicle is abstract with Car, Truck, Motorcycle subclasses"
- "Ticket is a POJO with ticketId, entryTime, vehicle, spot"
- "Level class maintains List and finds available spots"
- "NearestSpotStrategy, OptimalSpotStrategy implement ParkingStrategy interface"
- "Thread safety: synchronized methods + ConcurrentHashMap"

CONVERSATIONAL SCRIPT (How to approach in interview)

Phase 1: Requirements Clarification (5 mins)

You: "Let me start by clarifying the requirements. I'll break this into functional and non-functional requirements."

Functional Requirements:

- "The parking lot should have multiple levels"

- "It should support different vehicle types - bikes, cars, trucks"
- "Each vehicle type needs different spot sizes - compact, large, handicapped"
- "Entry and exit with ticket generation"
- "Pricing based on vehicle type and duration"
- "Display available spots at entrance"
- "Should we support hourly pricing or flat rate?"

Interviewer: "Hourly pricing. Keep it simple."

You: "Got it. For non-functional requirements:"

- "Thread safety - multiple vehicles can enter/exit simultaneously"
- "Scalability - system should handle multiple parking lots"
- "The system should be extensible - easy to add new vehicle types"

Interviewer: "Yes, focus on extensibility and thread safety."

Phase 2: Core Entities (5 mins)

You: "Let me identify the main entities:"

```
Core Classes:
1. ParkingLot (Singleton) – Main entry point
2. Level – Each floor in the parking lot
3. ParkingSpot (Abstract) – Base class for all spots
   – CompactSpot
   – LargeSpot
   – HandicappedSpot
   – BikeSpot
4. Vehicle (Abstract) – Base class for vehicles
   – Car
   – Truck
   – Motorcycle
5. Ticket – Generated on entry
6. Payment – Handles payment processing
7. ParkingStrategy (Interface) – How to find spots
8. PricingStrategy (Interface) – How to calculate fees
```

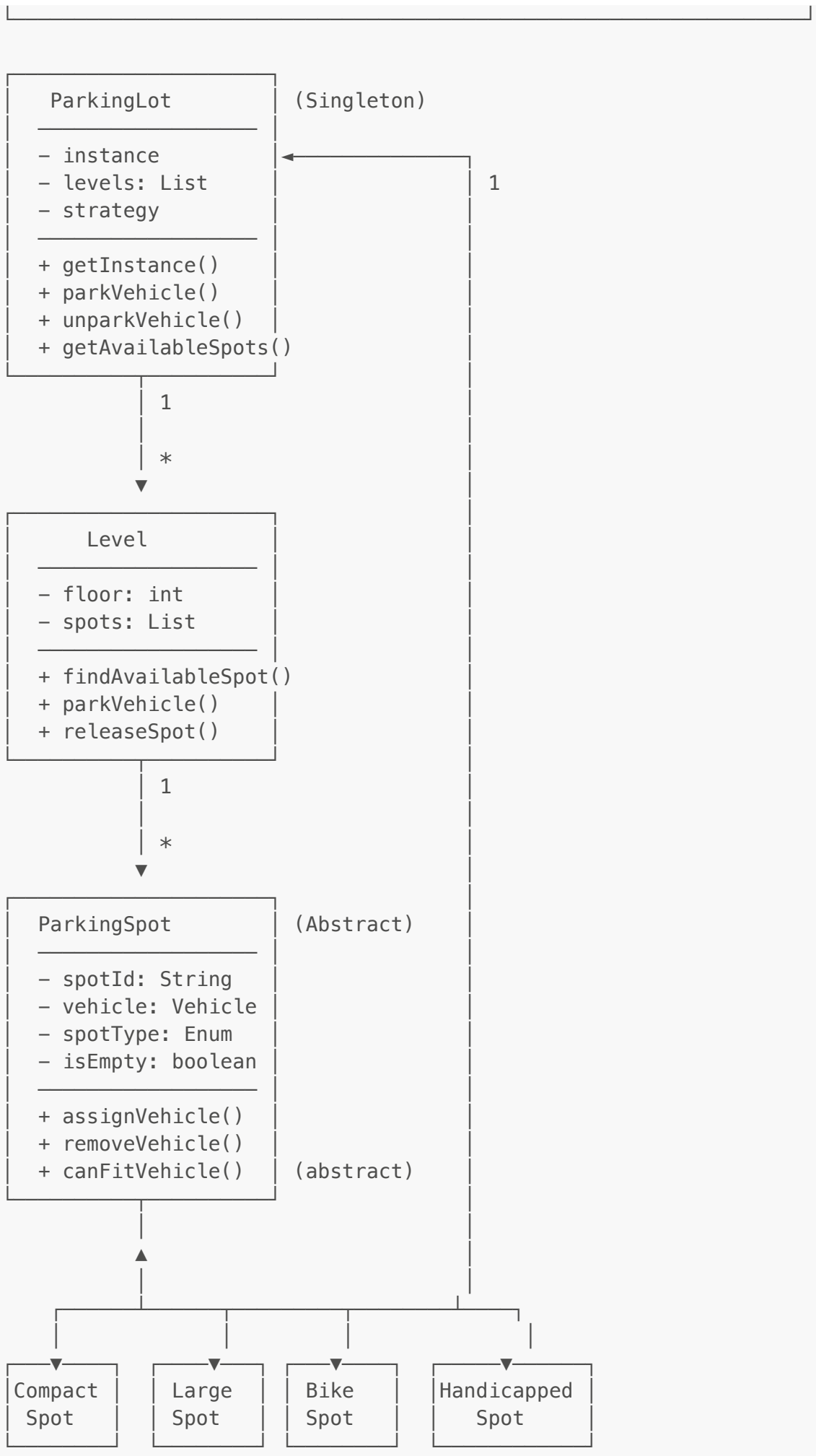
You: "Does this structure make sense? Any entities you think I'm missing?"

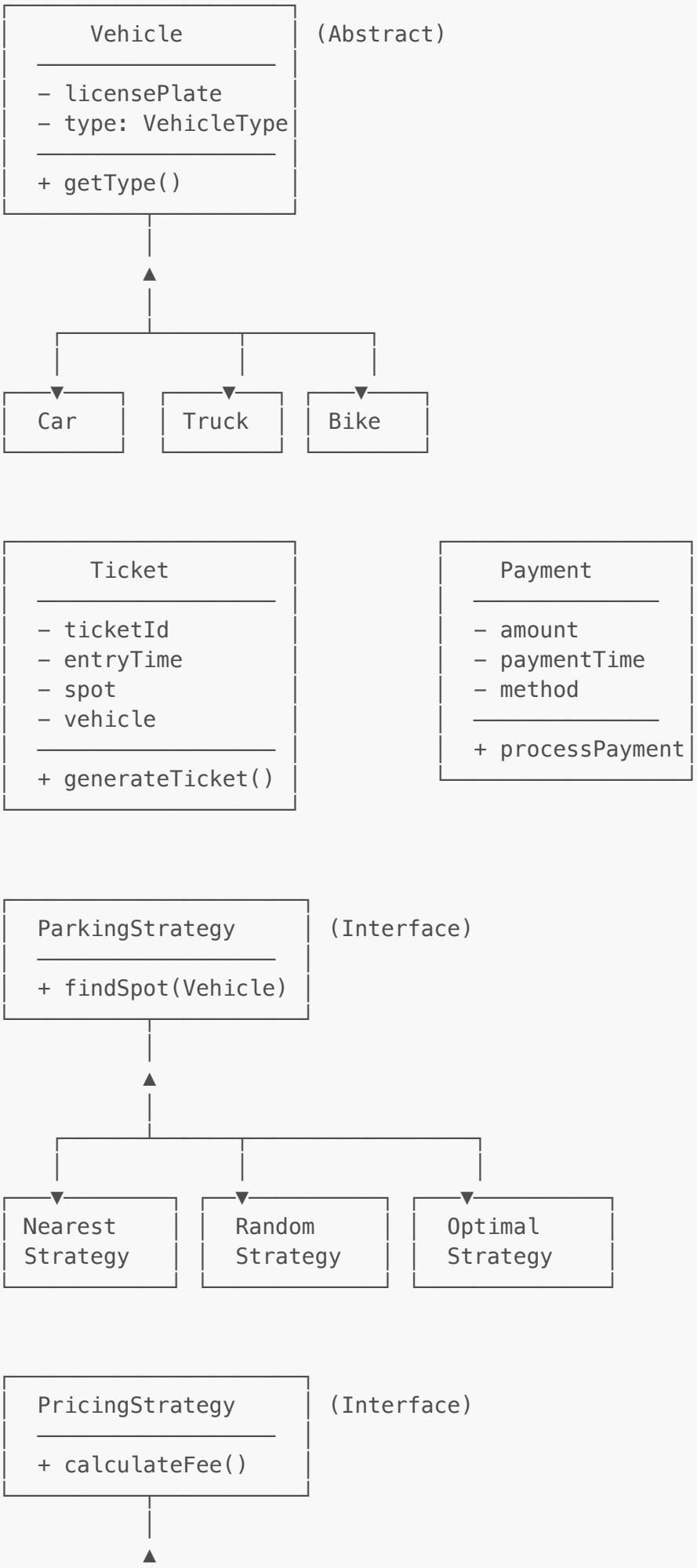
Interviewer: "Looks good. Proceed with design."

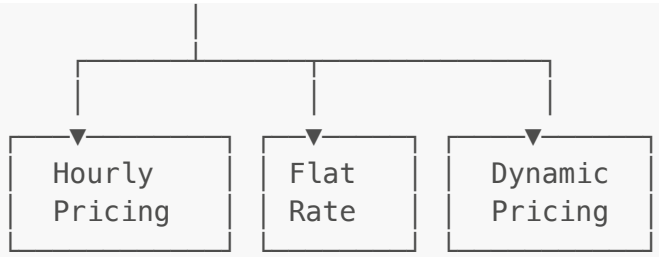
Phase 3: Class Diagram & Relationships (10 mins)

You: "Let me draw the class diagram with relationships:"

```
┌──────────────────────────────────────────────────────────────────────┐
│                        PARKING LOT SYSTEM                            │
```


ParkingLot
(Singleton)
- instance
- levels: List
- strategy
+ getInstance()
+ parkVehicle()
+ unparkVehicle()
+ getAvailableSpots()
1
1
*
Level
- floor: int
- spots: List
+ findAvailableSpot()
+ parkVehicle()
+ releaseSpot()
1
*
ParkingSpot
(Abstract)
- spotId: String
- vehicle: Vehicle
- spotType: Enum
- isEmpty: boolean
+ assignVehicle()
+ removeVehicle()
+ canFitVehicle()
(abstract)
Compact Spot
Large Spot
Bike Spot
Handicapped Spot


Vehicle
(Abstract)
- licensePlate
- type: VehicleType
+ getType()
Car
Truck
Bike
Ticket
- ticketId
- entryTime
- spot
- vehicle
+ generateTicket()
Payment
- amount
- paymentTime
- method
+ processPayment
ParkingStrategy
(Interface)
+ findSpot(Vehicle)
Nearest Strategy
Random Strategy
Optimal Strategy
PricingStrategy
(Interface)
+ calculateFee()



Phase 4: Design Patterns Used (3 mins)

You: "I'm using several design patterns here:"

1. **Singleton Pattern** - ParkingLot
    - "Only one instance should exist"
    - Thread-safe implementation needed
2. **Factory Pattern** - VehicleFactory, SpotFactory
    - "Creates different types of vehicles and spots"
3. **Strategy Pattern** - ParkingStrategy, PricingStrategy
    - "Allows different parking algorithms"
    - "Different pricing models"
4. **Observer Pattern** - DisplayBoard
    - "Notifies display boards when spots change"

Interviewer: "Good. Show me the implementation."

Phase 5: Core Implementation (15 mins)

You: "Let me implement the key classes:"

1. Enums

You: "Let me start with the basic enums to define our types. I'll create enums for vehicle types, spot types, and parking spot status. This makes the code type-safe and extensible."

```
public enum VehicleType {
    BIKE,
    CAR,
    TRUCK
}

public enum SpotType {
    BIKE_SPOT,
    COMPACT_SPOT,
```

```
    LARGE_SPOT,
    HANDICAPPED_SPOT
}

public enum ParkingSpotStatus {
    AVAILABLE,
    OCCUPIED,
    RESERVED
}
```

You: "These enums give us compile-time safety and make it easy to add new types later."

Interviewer: "Good. Show me the Vehicle hierarchy."

---

2. Vehicle (Abstract Class)

You: "Now I'll create an abstract Vehicle class. The key insight here is that all vehicles share common properties like license plate and type, but different vehicles might have different behaviors in the future - like size or parking requirements."

```
public abstract class Vehicle {
    private String licensePlate;
    private VehicleType type;

    public Vehicle(String licensePlate, VehicleType type) {
        this.licensePlate = licensePlate;
        this.type = type;
    }

    public String getLicensePlate() {
        return licensePlate;
    }

    public VehicleType getType() {
        return type;
    }
}

public class Car extends Vehicle {
    public Car(String licensePlate) {
        super(licensePlate, VehicleType.CAR);
    }
}

public class Motorcycle extends Vehicle {
    public Motorcycle(String licensePlate) {
        super(licensePlate, VehicleType.BIKE);
    }
}
```

```
public class Truck extends Vehicle {
    public Truck(String licensePlate) {
        super(licensePlate, VehicleType.TRUCK);
    }
}
```

You: "Notice I'm using simple concrete classes for Car, Motorcycle, and Truck. They just extend Vehicle and set their type. This follows the Open/Closed principle - if we need to add Electric Car or Bus later, we just create a new class without modifying existing code."

Interviewer: "Makes sense. What about parking spots?"

---

3. ParkingSpot (Abstract Class)

You: "Now the ParkingSpot class is where the core logic lives. I'm making it abstract because each spot type will have different rules for which vehicles can fit. Notice I'm using synchronized methods here - this is critical because multiple vehicles could try to park simultaneously in a multi-threaded environment."

```
public abstract class ParkingSpot {
    private String spotId;
    private SpotType spotType;
    private Vehicle vehicle;
    private ParkingSpotStatus status;

    public ParkingSpot(String spotId, SpotType spotType) {
        this.spotId = spotId;
        this.spotType = spotType;
        this.status = ParkingSpotStatus.AVAILABLE;
    }

    public synchronized boolean assignVehicle(Vehicle vehicle) {
        if (!canFitVehicle(vehicle)) {
            return false;
        }
        this.vehicle = vehicle;
        this.status = ParkingSpotStatus.OCCUPIED;
        return true;
    }

    public synchronized void removeVehicle() {
        this.vehicle = null;
        this.status = ParkingSpotStatus.AVAILABLE;
    }

    public abstract boolean canFitVehicle(Vehicle vehicle);

    public boolean isAvailable() {
        return status == ParkingSpotStatus.AVAILABLE;
    }
```

```
    // Getters
    public String getSpotId() { return spotId; }
    public SpotType getSpotType() { return spotType; }
    public Vehicle getVehicle() { return vehicle; }
}

public class CompactSpot extends ParkingSpot {
    public CompactSpot(String spotId) {
        super(spotId, SpotType.COMPACT_SPOT);
    }

    @Override
    public boolean canFitVehicle(Vehicle vehicle) {
        return vehicle.getType() == VehicleType.CAR ||
               vehicle.getType() == VehicleType.BIKE;
    }
}

public class LargeSpot extends ParkingSpot {
    public LargeSpot(String spotId) {
        super(spotId, SpotType.LARGE_SPOT);
    }

    @Override
    public boolean canFitVehicle(Vehicle vehicle) {
        return true; // Can fit any vehicle
    }
}

public class BikeSpot extends ParkingSpot {
    public BikeSpot(String spotId) {
        super(spotId, SpotType.BIKE_SPOT);
    }

    @Override
    public boolean canFitVehicle(Vehicle vehicle) {
        return vehicle.getType() == VehicleType.BIKE;
    }
}
```

You: "See how each concrete spot type implements `canFitVehicle()` with its own rules? CompactSpot can fit cars and bikes, LargeSpot can fit anything, and BikeSpot only fits bikes. This is the Strategy pattern in action - each spot has its own fitting strategy."

Interviewer: "Good. Show me the Ticket class."

---

4. Ticket

You: "The Ticket is a simple data holder that captures the parking session. When a vehicle enters, we generate a ticket with entry time, spot assigned, and vehicle details. Later we'll use this to calculate parking fees."

```java
import java.time.LocalDateTime;

public class Ticket {
    private String ticketId;
    private LocalDateTime entryTime;
    private ParkingSpot spot;
    private Vehicle vehicle;

    public Ticket(String ticketId, Vehicle vehicle, ParkingSpot spot) {
        this.ticketId = ticketId;
        this.vehicle = vehicle;
        this.spot = spot;
        this.entryTime = LocalDateTime.now();
    }

    // Getters
    public String getTicketId() { return ticketId; }
    public LocalDateTime getEntryTime() { return entryTime; }
    public ParkingSpot getSpot() { return spot; }
    public Vehicle getVehicle() { return vehicle; }
}
```

You: "It's immutable except for the getters, which is exactly what we want for a ticket."

Interviewer: "How do you organize spots across multiple floors?"

---

5. Level

You: "Great question! The Level class represents each floor in the parking lot. Here's the key design decision: I'm distributing spots by type - 50% compact, 30% large, 20% bike. In a real system, this ratio would be configurable. Also notice the `synchronized` on findAvailableSpot - this prevents race conditions when multiple threads search for spots."

```java
import java.util.*;

public class Level {
    private int floor;
    private List<ParkingSpot> spots;

    public Level(int floor, int numSpots) {
        this.floor = floor;
        this.spots = new ArrayList<>();

        // Initialize spots (example: 50% compact, 30% large, 20% bike)
        int compact = (int)(numSpots * 0.5);
        int large = (int)(numSpots * 0.3);
        int bike = numSpots - compact - large;

        for (int i = 0; i < compact; i++) {
```

```
            spots.add(new CompactSpot("L" + floor + "-C" + i));
        }
        for (int i = 0; i < large; i++) {
            spots.add(new LargeSpot("L" + floor + "-L" + i));
        }
        for (int i = 0; i < bike; i++) {
            spots.add(new BikeSpot("L" + floor + "-B" + i));
        }
    }

    public synchronized ParkingSpot findAvailableSpot(Vehicle vehicle) {
        for (ParkingSpot spot : spots) {
            if (spot.isAvailable() && spot.canFitVehicle(vehicle)) {
                return spot;
            }
        }
        return null;
    }

    public int getAvailableSpotCount() {
        return (int)
spots.stream().filter(ParkingSpot::isAvailable).count();
    }

    public int getFloor() { return floor; }
}
```

You: "The level encapsulates all spots on that floor and provides a clean interface to find available spots."

Interviewer: "I see you mentioned Strategy pattern earlier. Show me how you implement different parking strategies."

---

6. Parking Strategy (Strategy Pattern)

You: "Excellent! This is where the Strategy pattern really shines. Different parking lots might want different algorithms - find the nearest spot, find the optimal level, or even randomize. By creating a ParkingStrategy interface, we can swap algorithms at runtime without changing any core logic."

You: "Let me show you two concrete strategies:"

```
public interface ParkingStrategy {
    ParkingSpot findSpot(Vehicle vehicle, List<Level> levels);
}

public class NearestSpotStrategy implements ParkingStrategy {
    @Override
    public ParkingSpot findSpot(Vehicle vehicle, List<Level> levels) {
        // Find spot on the nearest level (lowest floor number)
        for (Level level : levels) {
            ParkingSpot spot = level.findAvailableSpot(vehicle);
```

```
            if (spot != null) {
                return spot;
            }
        }
        return null;
    }
}

public class OptimalSpotStrategy implements ParkingStrategy {
    @Override
    public ParkingSpot findSpot(Vehicle vehicle, List<Level> levels) {
        // Find level with most available spots
        Level optimalLevel = null;
        int maxAvailable = 0;

        for (Level level : levels) {
            int available = level.getAvailableSpotCount();
            if (available > maxAvailable) {
                maxAvailable = available;
                optimalLevel = level;
            }
        }

        return optimalLevel != null ?
               optimalLevel.findAvailableSpot(vehicle) : null;
    }
}
```

You: "The NearestSpotStrategy finds the first available spot starting from the lowest floor - great for user convenience. The OptimalSpotStrategy distributes vehicles evenly by choosing the level with most available spots - better for load balancing. The beauty is, I can add a RandomStrategy or PremiumStrategy later without touching existing code."

Interviewer: "And how do you handle pricing?"

---

7. Pricing Strategy (Strategy Pattern)

You: "Same pattern! PricingStrategy interface with different implementations. Let me show you hourly pricing, which is most common:"

```
import java.time.Duration;
import java.time.LocalDateTime;

public interface PricingStrategy {
    double calculateFee(Ticket ticket);
}

public class HourlyPricingStrategy implements PricingStrategy {
    private static final double BIKE_RATE = 10.0;
    private static final double CAR_RATE = 20.0;
```

```
    private static final double TRUCK_RATE = 30.0;

    @Override
    public double calculateFee(Ticket ticket) {
        Duration duration = Duration.between(
            ticket.getEntryTime(),
            LocalDateTime.now()
        );

        long hours = duration.toHours();
        if (hours == 0) hours = 1; // Minimum 1 hour

        VehicleType type = ticket.getVehicle().getType();
        double hourlyRate;

        switch (type) {
            case BIKE:
                hourlyRate = BIKE_RATE;
                break;
            case CAR:
                hourlyRate = CAR_RATE;
                break;
            case TRUCK:
                hourlyRate = TRUCK_RATE;
                break;
            default:
                hourlyRate = CAR_RATE;
        }

        return hours * hourlyRate;
    }
}
```

You: "The key here is calculating duration from entry time to now, then applying the rate based on vehicle type. Bikes are cheapest, trucks are most expensive. I'm also ensuring a minimum of 1 hour charge even if someone exits within minutes - that's a business rule."

You: "We could easily add FlatRatePricing, PeakHourPricing, or SubscriptionPricing by implementing the same interface."

Interviewer: "Now tie it all together with the main ParkingLot class."

---

8. ParkingLot (Singleton Pattern)

You: "This is the heart of the system! ParkingLot is a Singleton because we want exactly one instance managing all operations. Let me walk through the key methods:"

You: "First, the Singleton setup with thread-safe lazy initialization:"

```
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;

public class ParkingLot {
    private static ParkingLot instance;
    private List<Level> levels;
    private Map<String, Ticket> activeTickets;
    private ParkingStrategy parkingStrategy;
    private PricingStrategy pricingStrategy;

    // Private constructor for Singleton
    private ParkingLot() {
        this.levels = new ArrayList<>();
        this.activeTickets = new ConcurrentHashMap<>();
        this.parkingStrategy = new NearestSpotStrategy();
        this.pricingStrategy = new HourlyPricingStrategy();
    }

    // Thread-safe Singleton
    public static synchronized ParkingLot getInstance() {
        if (instance == null) {
            instance = new ParkingLot();
        }
        return instance;
    }

    public void addLevel(Level level) {
        levels.add(level);
    }

    public void setParkingStrategy(ParkingStrategy strategy) {
        this.parkingStrategy = strategy;
    }

    public void setPricingStrategy(PricingStrategy strategy) {
        this.pricingStrategy = strategy;
    }

    // Park Vehicle
    public synchronized Ticket parkVehicle(Vehicle vehicle) {
        ParkingSpot spot = parkingStrategy.findSpot(vehicle, levels);

        if (spot == null) {
            System.out.println("No available spot for " +
vehicle.getType());
            return null;
        }

        spot.assignVehicle(vehicle);
        String ticketId = UUID.randomUUID().toString();
        Ticket ticket = new Ticket(ticketId, vehicle, spot);
        activeTickets.put(ticketId, ticket);
```

```
        System.out.println("Vehicle parked: " + vehicle.getLicensePlate() +
                           " at spot: " + spot.getSpotId());
        return ticket;
    }

    // Unpark Vehicle
    public synchronized double unparkVehicle(String ticketId) {
        Ticket ticket = activeTickets.get(ticketId);

        if (ticket == null) {
            System.out.println("Invalid ticket");
            return 0;
        }

        double fee = pricingStrategy.calculateFee(ticket);
        ParkingSpot spot = ticket.getSpot();
        spot.removeVehicle();
        activeTickets.remove(ticketId);

        System.out.println("Vehicle unparked: " +
                           ticket.getVehicle().getLicensePlate() +
                           " | Fee: $" + fee);
        return fee;
    }

    public void displayAvailability() {
        System.out.println("\n=== Parking Availability ===");
        for (Level level : levels) {
            System.out.println("Level " + level.getFloor() +
                               ": " + level.getAvailableSpotCount() +
                               " spots available");
        }
        System.out.println("============================\n");
    }
}
```

You: "Let me explain the critical parts:"

You: "**parkVehicle()** - This is synchronized to prevent race conditions. The flow is: use the parking strategy to find a spot, assign the vehicle to that spot, generate a ticket with unique ID, and store it in our active tickets map. We're using ConcurrentHashMap for thread safety."

You: "**unparkVehicle()** - Also synchronized. We retrieve the ticket, calculate the fee using our pricing strategy, release the spot, and remove the ticket from active tickets. Notice we're returning the fee - in a real system, this would integrate with a payment gateway."

You: "The key insight is that ParkingLot acts as a Facade - it hides all the complexity of levels, spots, strategies, and provides a simple interface: park and unpark."

Interviewer: "Great! Show me how this would be used."

Phase 6: Usage Example (3 mins)

You: "Let me demonstrate a complete flow - from creating the parking lot to parking and unparking vehicles:"

```
public class ParkingLotDemo {
    public static void main(String[] args) {
        // Get parking lot instance
        ParkingLot parkingLot = ParkingLot.getInstance();

        // Add 3 levels, each with 100 spots
        parkingLot.addLevel(new Level(1, 100));
        parkingLot.addLevel(new Level(2, 100));
        parkingLot.addLevel(new Level(3, 100));

        // Display initial availability
        parkingLot.displayAvailability();

        // Create vehicles
        Vehicle car1 = new Car("KA01-1234");
        Vehicle bike1 = new Motorcycle("KA02-5678");
        Vehicle truck1 = new Truck("KA03-9012");

        // Park vehicles
        Ticket ticket1 = parkingLot.parkVehicle(car1);
        Ticket ticket2 = parkingLot.parkVehicle(bike1);
        Ticket ticket3 = parkingLot.parkVehicle(truck1);

        // Display availability after parking
        parkingLot.displayAvailability();

        // Simulate some time passing
        try {
            Thread.sleep(2000); // 2 seconds (simulating 2 hours)
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        // Unpark vehicles
        if (ticket1 != null) {
            double fee = parkingLot.unparkVehicle(ticket1.getTicketId());
            System.out.println("Car fee: $" + fee);
        }

        if (ticket2 != null) {
            double fee = parkingLot.unparkVehicle(ticket2.getTicketId());
            System.out.println("Bike fee: $" + fee);
        }

        // Display final availability
        parkingLot.displayAvailability();
```

```
        // Test with different strategy
        System.out.println("\n=== Switching to Optimal Strategy ===");
        parkingLot.setParkingStrategy(new OptimalSpotStrategy());

        Vehicle car2 = new Car("KA04-3456");
        Ticket ticket4 = parkingLot.parkVehicle(car2);
    }
}
```

You: "In this demo, I'm:"

- "Creating a parking lot with 3 levels, 100 spots each"
- "Parking different vehicle types - car, bike, truck"
- "Simulating time passing with Thread.sleep"
- "Unparking vehicles and calculating fees"
- "Demonstrating strategy switching at runtime"

You: "The output would show which spots were assigned, fees charged, and availability at each step."

Interviewer: "How do you handle concurrent access and edge cases?"

Phase 7: Handling Edge Cases (3 mins)

You: "Excellent question! Let me address the key edge cases:"

1. Thread Safety:

You: "This is crucial because in the real world, multiple vehicles will try to enter simultaneously. I've handled this at multiple levels:"

```
// All critical methods are synchronized
public synchronized Ticket parkVehicle(Vehicle vehicle) {
    // Thread-safe parking
}

// Spot assignment is also synchronized
public synchronized boolean assignVehicle(Vehicle vehicle) {
    // Thread-safe spot assignment
}

// Using ConcurrentHashMap for active tickets
private Map<String, Ticket> activeTickets = new ConcurrentHashMap<>();
```

You: "By synchronizing at three levels - the main operations in ParkingLot, the spot assignment, and using thread-safe collections - we prevent race conditions like two vehicles getting the same spot."

2. No Available Spots:

You: "What happens when the parking lot is full? We handle this gracefully:"

```
public synchronized Ticket parkVehicle(Vehicle vehicle) {
    ParkingSpot spot = parkingStrategy.findSpot(vehicle, levels);

    if (spot == null) {
        System.out.println("Parking lot is full");
        return null;
    }
    // ... rest of code
}
```

You: "Instead of crashing, we return null and log a message. The calling code can then inform the customer that the lot is full."

3. Invalid Ticket:

You: "What if someone tries to exit with a fake or already-used ticket?"

```
public synchronized double unparkVehicle(String ticketId) {
    Ticket ticket = activeTickets.get(ticketId);

    if (ticket == null) {
        throw new IllegalArgumentException("Invalid ticket ID");
    }
    // ... rest of code
}
```

You: "We validate the ticket exists before processing. In production, you might throw a custom exception instead."

4. Multiple Parking Lots:

You: "The Singleton pattern limits us to one parking lot instance. What if we need to manage multiple locations - airport, mall, hotel? Here's how I'd refactor:"

```
// Instead of Singleton, use Factory pattern
public class ParkingLotFactory {
    private Map<String, ParkingLot> parkingLots = new HashMap<>();

    public ParkingLot getParkingLot(String locationId) {
        return parkingLots.computeIfAbsent(locationId,
            k -> new ParkingLot(locationId));
    }
}
```

You: "This Factory pattern maintains separate instances per location while keeping the same clean interface."

Interviewer: "Interesting! Now let me ask some follow-up questions."

Phase 8: Follow-up Questions & Answers (5 mins)

Interviewer: "How would you handle reserved parking?"

You: "Great question! Reserved parking is a common requirement - think handicapped spots or executive parking. I'd extend the ParkingSpot class:"

```
public class ReservedSpot extends ParkingSpot {
    private String reservedForLicensePlate;

    @Override
    public boolean canFitVehicle(Vehicle vehicle) {
        if (reservedForLicensePlate == null) {
            return super.canFitVehicle(vehicle);
        }
        return vehicle.getLicensePlate().equals(reservedForLicensePlate);
    }
}
```

You: "The beauty is I don't need to change any existing code - just add this new spot type. That's the power of the Open/Closed principle."

Interviewer: "How would you add valet parking?"

You: "Valet is interesting because it adds a service layer on top. The customer doesn't find their own spot - the valet does. Here's how I'd model it:"

```
public class ValetService {
    private ParkingLot parkingLot;
    private Queue<Vehicle> waitingQueue = new LinkedList<>();

    public Ticket acceptVehicle(Vehicle vehicle) {
        Ticket ticket = parkingLot.parkVehicle(vehicle);
        if (ticket == null) {
            waitingQueue.offer(vehicle);
            return generateWaitingTicket(vehicle);
        }
        return ticket;
    }
}
```

You: "The ValetService acts as a wrapper - if no spot is available, we queue the vehicle and give them a waiting ticket. When a spot opens up, we can process the queue."

Interviewer: "How would you implement a payment system?"

You: "Payment processing is another layer we'd add. I'd use the Strategy pattern again for different payment methods:"

```
public interface PaymentProcessor {
    boolean processPayment(double amount, PaymentMethod method);
}

public enum PaymentMethod {
    CASH, CARD, UPI, WALLET
}

public class PaymentService implements PaymentProcessor {
    @Override
    public boolean processPayment(double amount, PaymentMethod method) {
        switch (method) {
            case CARD:
                return processCardPayment(amount);
            case UPI:
                return processUPIPayment(amount);
            default:
                return true;
        }
    }
}
```

You: "Each payment method would have its own processing logic. In a real system, this would integrate with payment gateways like Stripe, Razorpay, or banking APIs. The key is that the ParkingLot class doesn't need to know HOW payment is processed - it just calls processPayment() and gets a success/failure response."

Interviewer: "Excellent! You've covered a lot. Any final thoughts?"

You: "Yes! The key takeaways are:"

- "**Design Patterns**: Singleton for single instance, Strategy for pluggable algorithms, Factory for object creation, Abstract classes for common behavior"
- "**SOLID Principles**: Each class has one responsibility, open for extension but closed for modification, depend on interfaces not implementations"
- "**Thread Safety**: Critical in a real-world parking system where concurrent access is guaranteed"
- "**Extensibility**: This design easily extends to hotel booking, movie tickets, restaurant reservations - it's the same core problem"

SOLID PRINCIPLES IN DEPTH

You: "Let me explain how SOLID principles are applied in this design. Understanding these is crucial for any LLD interview."

---

1. Single Responsibility Principle (SRP)

Purpose: Each class should have only ONE reason to change - one job, one responsibility.

Problem it solves: Without SRP, you end up with "God classes" that do everything:

```
// BAD: Vehicle class doing too much
class Vehicle {
    private String plate;

    // Vehicle data
    public void park() { ... }           // Parking logic
    public void calculateFee() { ... }   // Payment logic
    public void sendEmail() { ... }      // Notification logic
    public void display() { ... }        // UI logic
}
// If payment logic changes, you modify Vehicle class!
```

Advantages:

- ✅ **Easier to understand** - Each class has a clear purpose
- ✅ **Easier to maintain** - Changes are isolated
- ✅ **Better testability** - Test one responsibility at a time
- ✅ **Team scalability** - Different devs work on different classes

In our design:

```
// GOOD: Separated responsibilities
class Vehicle {
    // ONLY stores vehicle data
}

class ParkingSpot {
    // ONLY manages spot availability
}

class PricingStrategy {
    // ONLY calculates fees
}

class Ticket {
    // ONLY stores parking session data
}
```

Interview tip: "If I need to change how pricing works, I only touch `PricingStrategy`. If I add a new vehicle type, I only add a new `Vehicle` subclass. Each change is localized."

---

2. Open/Closed Principle (OCP)

Purpose: Classes should be OPEN for extension but CLOSED for modification.

Problem it solves: Without OCP, every new feature requires modifying existing code:

```
// BAD: Need to modify existing code for new features
class ParkingLot {
    public double calculateFee(Vehicle v, long hours) {
        if (v.type == "CAR") return hours * 20;
        else if (v.type == "BIKE") return hours * 10;
        else if (v.type == "TRUCK") return hours * 30;
        // To add BUS, you modify this method - RISKY!
    }
}
```

Advantages:

- ✅ **Zero regression risk** - Existing code stays untouched
- ✅ **Easy to extend** - Add features by adding new classes
- ✅ **Parallel development** - Multiple devs add features simultaneously
- ✅ **Stable codebase** - Core logic never changes

In our design:

```
// GOOD: Extend by adding new classes
interface PricingStrategy {
    double calculateFee(Ticket ticket);
}

class HourlyPricing implements PricingStrategy { ... }
class FlatRatePricing implements PricingStrategy { ... }
class PeakHourPricing implements PricingStrategy { ... }  // NEW - no
modification!

// Adding new vehicle type:
abstract class Vehicle { ... }

class Car extends Vehicle { ... }
class Bike extends Vehicle { ... }
class Bus extends Vehicle { ... }  // NEW - no modification to existing
classes!
```

Interview tip: "To add peak hour pricing, I create a new `PeakHourPricing` class. Zero changes to existing code. To add electric car, I create `ElectricCar extends Vehicle`. The system is closed for modification but open for extension."

---

3. Liskov Substitution Principle (LSP)

Purpose: Subclasses must be substitutable for their parent classes without breaking the program.

Problem it solves: Without LSP, subclasses violate parent contracts:

```
// BAD: Violates LSP
class ParkingSpot {
    public boolean assignVehicle(Vehicle v) {
        // Always returns true if space available
    }
}

class ReadOnlySpot extends ParkingSpot {
    @Override
    public boolean assignVehicle(Vehicle v) {
        throw new UnsupportedOperationException();  // BREAKS CONTRACT!
    }
}

// Code expecting ParkingSpot behavior will crash!
ParkingSpot spot = new ReadOnlySpot();
spot.assignVehicle(car);  // BOOM! Exception
```

Advantages:

- ✅ **Predictable behavior** - Subclasses behave as expected
- ✅ **Polymorphism works** - Can use parent type everywhere
- ✅ **No surprises** - Substituting subclass doesn't break code
- ✅ **Code reusability** - Write once, works for all subtypes

In our design:

```
// GOOD: All subclasses honor the contract
abstract class Vehicle {
    public VehicleType getType();  // All subclasses implement this
}

class Car extends Vehicle {
    @Override
    public VehicleType getType() { return VehicleType.CAR; }  // ✓ Works
}

class Bike extends Vehicle {
    @Override
    public VehicleType getType() { return VehicleType.BIKE; }  // ✓ Works
}

// Polymorphism works perfectly:
Vehicle v = new Car();  // Or new Bike() or new Truck()
ParkingSpot spot = level.findAvailableSpot(v);  // Works for ANY vehicle
subclass
```

Interview tip: "Any code that works with `Vehicle` will work with `Car`, `Bike`, or `Truck` without modifications. If I pass a `CompactSpot` where `ParkingSpot` is expected, it works perfectly because it

honors all contracts."

---

4. Interface Segregation Principle (ISP)

Purpose: Clients should not be forced to depend on interfaces they don't use.

Problem it solves: Without ISP, interfaces become bloated:

```
// BAD: Fat interface forces unnecessary implementations
interface ParkingOperations {
    Ticket parkVehicle(Vehicle v);
    double unparkVehicle(String ticketId);
    void addLevel(Level l);
    void displayAvailability();
    void generateReport();           // Not all clients need this
    void sendNotification();         // Not all clients need this
    void processPayment();           // Not all clients need this
}

// DisplayBoard only needs displayAvailability() but must implement ALL
methods!
class DisplayBoard implements ParkingOperations {
    @Override
    public void generateReport() { throw new
UnsupportedOperationException(); }  // Forced!
}
```

Advantages:

- ✅ **Lean interfaces** - Only the methods you need
- ✅ **Better cohesion** - Related methods grouped together
- ✅ **Easier to implement** - No dummy implementations
- ✅ **Decoupled code** - Changes don't ripple unnecessarily

In our design:

```
// GOOD: Segregated interfaces
interface ParkingStrategy {
    ParkingSpot findSpot(Vehicle vehicle, List<Level> levels);
    // ONLY parking logic - nothing else
}

interface PricingStrategy {
    double calculateFee(Ticket ticket);
    // ONLY pricing logic - nothing else
}

interface PaymentProcessor {
    boolean processPayment(double amount, PaymentMethod method);
```

```
    // ONLY payment logic - nothing else
}

// Each client depends ONLY on what it needs:
class NearestSpotStrategy implements ParkingStrategy {
    // Only implements findSpot() - nothing else
}

class HourlyPricing implements PricingStrategy {
    // Only implements calculateFee() - nothing else
}
```

Interview tip: "I keep interfaces focused. `ParkingStrategy` only has `findSpot()`. `PricingStrategy` only has `calculateFee()`. Clients depend only on what they need, nothing more."

---

5. Dependency Inversion Principle (DIP)

Purpose: High-level modules should not depend on low-level modules. Both should depend on abstractions.

Problem it solves: Without DIP, high-level code is tightly coupled to low-level implementation:

```
// BAD: ParkingLot directly depends on concrete classes
class ParkingLot {
    private HourlyPricing pricing = new HourlyPricing();  // TIGHT
COUPLING!
    private NearestSpotStrategy strategy = new NearestSpotStrategy();  //
TIGHT COUPLING!

    public double unparkVehicle(String ticketId) {
        // If you want to change pricing, you must modify ParkingLot class!
        return pricing.calculateFee(ticket);
    }
}
```

Advantages:

- ✅ **Loose coupling** - Easy to swap implementations
- ✅ **Testability** - Can inject mocks for testing
- ✅ **Flexibility** - Change behavior at runtime
- ✅ **Maintainability** - Changes in low-level don't affect high-level

In our design:

```
// GOOD: Depend on abstractions (interfaces)
class ParkingLot {
    private PricingStrategy pricingStrategy;      // Interface, not
concrete class
```

```
    private ParkingStrategy parkingStrategy;      // Interface, not
concrete class

    // Dependency Injection via setter
    public void setPricingStrategy(PricingStrategy strategy) {
        this.pricingStrategy = strategy;
    }

    public void setParkingStrategy(ParkingStrategy strategy) {
        this.parkingStrategy = strategy;
    }

    public double unparkVehicle(String ticketId) {
        // Uses interface - don't care about concrete implementation
        return pricingStrategy.calculateFee(ticket);
    }
}

// At runtime, inject any implementation:
ParkingLot lot = ParkingLot.getInstance();
lot.setPricingStrategy(new HourlyPricing());      // Can change to FlatRate
anytime
lot.setParkingStrategy(new OptimalStrategy());    // Can change to Nearest
anytime
```

Interview tip: "ParkingLot doesn't know if it's using HourlyPricing or FlatRatePricing - it just calls `calculateFee()` on the interface. I can swap strategies at runtime without modifying ParkingLot. For testing, I can inject mock strategies."

KEY TAKEAWAYS

Design Patterns Used:

✅ **Singleton** - ParkingLot (single instance) ✅ **Factory** - Vehicle creation, Spot creation ✅ **Strategy** - Parking algorithms, Pricing models ✅ **Abstract Class** - Vehicle, ParkingSpot (code reuse)

SOLID Principles Applied:

✅ **Single Responsibility (SRP)** - Vehicle stores data, ParkingSpot manages availability, PricingStrategy calculates fees ✅ **Open/Closed (OCP)** - Add new vehicle/pricing types without modifying existing code ✅ **Liskov Substitution (LSP)** - Car, Bike, Truck can substitute Vehicle anywhere ✅ **Interface Segregation (ISP)** - Small focused interfaces: ParkingStrategy, PricingStrategy ✅ **Dependency Inversion (DIP)** - ParkingLot depends on Strategy interfaces, not concrete implementations

Thread Safety:

✅ Synchronized methods for critical operations ✅ ConcurrentHashMap for shared state ✅ Atomic operations where needed

Extensibility:

✅ Easy to add new vehicle types ✅ Easy to add new spot types ✅ Easy to change parking/pricing strategies ✅ Easy to add new features (valet, reservation)

COMMON MISTAKES TO AVOID

❌ Not making ParkingLot thread-safe ❌ Hard-coding vehicle/spot types ❌ Not using design patterns ❌ Poor separation of concerns ❌ Not handling edge cases (full lot, invalid ticket) ❌ Forgetting to release spots on exit ❌ Not considering extensibility

---

TIME COMPLEXITY

Operation	Complexity	Notes
Park Vehicle	O(n)	n = number of spots per level
Unpark Vehicle	O(1)	Direct ticket lookup
Find Spot	O(n)	Linear search through spots
Display Availability	O(m)	m = number of levels

Optimization: Use a Min-Heap to track available spots for O(log n) insertion.

END OF PARKING LOT SYSTEM GUIDE

This covers **90%** of Hotel Booking, Movie Tickets, Car Rental, Restaurant Booking variations!